Deep Learning - COSC2779

Representation Learning

Dr. Ruwan Tennakoon

RMIT UNIVERSITY

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

Problems in Applying Deep Learning

When applying deep learning in practice:

- Usually, **deep network** architectures are required for **complex tasks**.
- They have many parameters and **need large labeled datasets** to obtain good generalization.
- Generating large "labeled" dataset for your task is difficult.
  - **Time consuming**: You may want to complete the project withing tight deadlines.
  - **Expensive**: Annotating a large dataset is expensive (infrastructure and manpower).
  - **Not possible**: e.g. in some medical imaging tasks, it is not ethical to put a healthy person through imaging or prevalence can be low.

Deep learning can only be applied when large datasets are available.

Problems in Applying Deep Learning

When applying deep learning in practice:

- Usually, **deep network** architectures are required for **complex tasks**.
- They have many parameters and **need large labeled datasets** to obtain good generalization.
- Generating large "labeled" dataset for your task is difficult.
  - **Time consuming**: You may want to complete the project withing tight deadlines.
  - **Expensive**: Annotating a large dataset is expensive (infrastructure and manpower).
  - **Not possible**: e.g. in some medical imaging tasks, it is not ethical to put a healthy person through imaging or prevalence can be low.

~~Deep learning can only be applied when large labeled datasets are available.~~
Not correct! There are alternatives.

Objective of the lecture

Explore practical approaches used in deep learning that can handle limited data availability:

- Relatively small labeled dataset only: **Transfer learning**.
- Large unlabelled data: **Self supervised learning**.

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

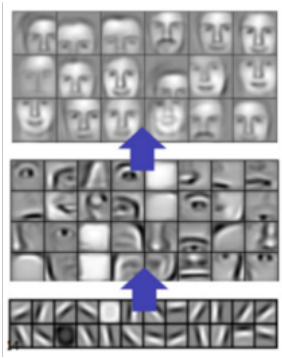


Image from: Paper

Learned features from a face recognition task.

Observation:

- Layers towards the bottom (close to input) learn *general features*.
- Layers towards the top (close to output) learn task *specific features*.

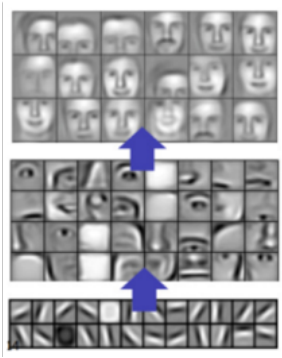


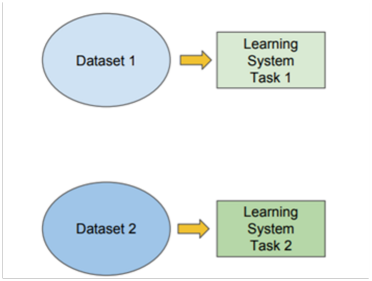
Image from: Paper

Learned features from a face recognition task.

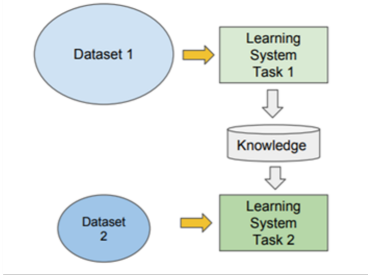
Observation:

- Layers towards the bottom (close to input) learn *general features*.
- Layers towards the top (close to output) learn task *specific features*.

Can we take the learned features on one task and apply them to a new task?



Traditional



Transfer Learning

Outline

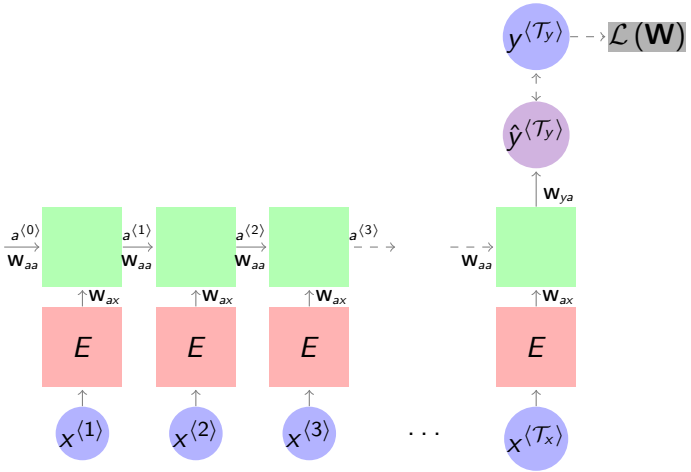
1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

Assume you are hired by RMIT to design a system that automatically mark an email coming into the inbox of RMIT academics as: “Urgent”, “Non-Urgent”.

The training dataset is 2000 emails with correct annotation.

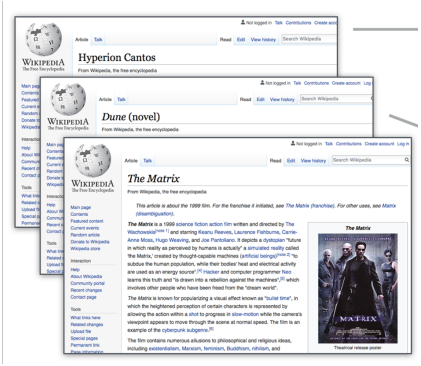


Need an RNN to capture the information in the language structure.

Not enough examples to learn the complexities and semantics of language.

- Find a very large dataset that has **similar** language structure, train a word embedding matrix there (or take pre-trained models).
- Transfer what is learned to your dataset.

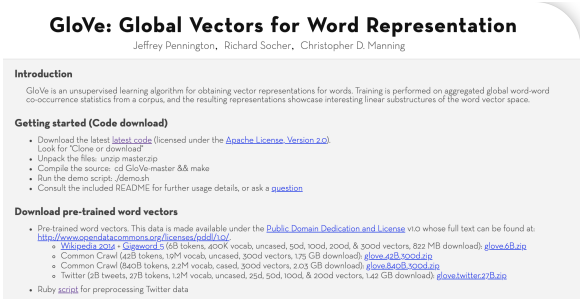
Why would this word? Word embeddings learn the semantics in the language. The knowledge gained will transfer to other tasks.



Option 1:

Step 1.1: Find a large text corpus related to the task. e.g. Spam/no-spam email dataset, Wikepedia, Twitter.

Step 1.2: Learn embedding matrix, E, using techniques discussed last week. e.g. language modelling (predict next word).



GloVe: Global Vectors for Word Representation

Jeffrey Pennington, Richard Socher, Christopher D. Manning

Introduction

GloVe is an unsupervised learning algorithm for obtaining vector representations for words. Training is performed on aggregated global word-word co-occurrence statistics from a corpus, and the resulting representations showcase interesting linear substructures of the word vector space.

Getting started (Code download)

- Download the latest latest code (licensed under the Apache License, Version 2.0).
  Look for "Clone or download"
- Unpack the files: unzip master.zip
- Compile the source: cd GloVe-master && make
- Run the demo script: ./demo.sh
- Consult the included README for further usage details, or ask a question

Download pre-trained word vectors

- Pre-trained word vectors. This data is made available under the Public Domain Dedication and License v1.0 whose full text can be found at: http://www.opendatacommons.org/licenses/pddl/1.0/.
  - Wikipedia 2014 + Gigaword 5 (6B tokens, 400K vocab, uncased, 50d, 100d, 200d, & 300d vectors, 822 MB download): glove.6B.zip
  - Common Crawl (42B tokens, 1.9M vocab, uncased, 300d vectors, 1.75 GB download): glove.42B.300d.zip
  - Common Crawl (840B tokens, 2.2M vocab, cased, 300d vectors, 2.03 GB download): glove.840B.300d.zip
  - Twitter (2B tweets, 27B tokens, 1.2M vocab, uncased, 25d, 50d, 100d, & 200d vectors, 1.42 GB download): glove.twitter.27B.zip
- Ruby script for preprocessing Twitter data

Option 2:

Step 1: Download pre-learned word embeddings.

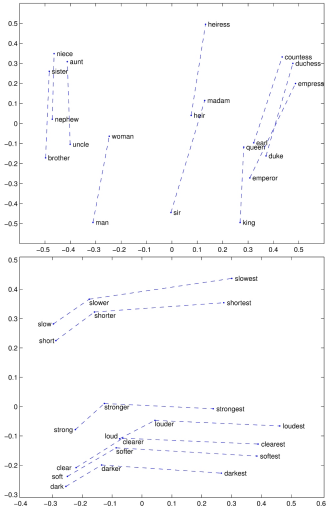
- Word2Vec
- GloVe

Option 2 is the most popular.

Pre-trained embedding matrices (E_{glove}) available as `.txt` files.

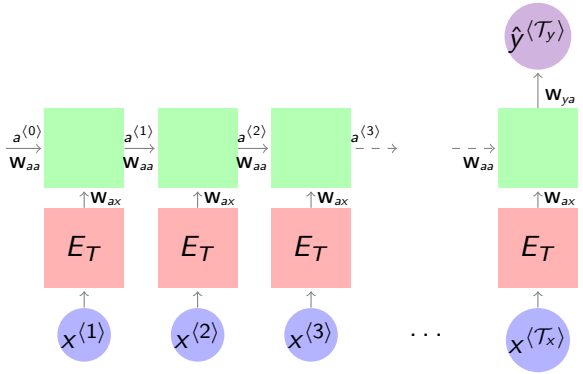
This data is made available under the Public Domain Dedication and License v1.0

- Wikipedia 2014 + Gigaword 5 (400K vocab, uncased, 50d, 100d, 200d, & 300d vectors)
- Common Crawl (1.9M vocab, uncased, 300d vectors)
- Common Crawl (2.2M vocab, cased, 300d vectors)
- Twitter (1.2M vocab, uncased, 25d, 50d, 100d, & 200d vectors)



Step 2: Map the vocabulary of word embedding to vocabulary of the task.

1. Tokenize the training set of the task to get the task-vocabulary or select a suitable vocabulary.
2. Map each word of the task-vocabulary to embedding-vocabulary.
3. Compose the embedding matrix (E_T) that has number of rows equal to the number of words in the task-vocabulary.



Step 3: Plug in the E_T to the embedding layer.

Step 4: Freeze the Embedding layer and train the RNN part.

Step 5: (optional) Fine-tune the embedding. Unfreeze embedding layer and train for few more epoch with a smaller learning rate.

- The Illustrated BERT, ELMo, and co. - How NLP Cracked Transfer Learning
- Efficient multi-lingual language model fine-tuning

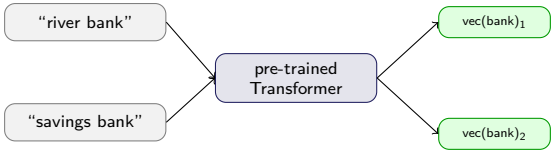
From static to contextual word vectors

Word2Vec / GloVe give **one fixed vector per word**, regardless of context. So the two senses of *bank* get the *same* vector:

- "river **bank**" and "savings **bank**"

Pre-trained Transformers (ELMo → BERT / GPT) instead produce **contextual** embeddings: the vector for a word is *computed from the whole sentence*, so the same word gets different vectors in different contexts.

This shift – from a fixed lookup table to a **pre-trained network that reads the sentence** – is what made NLP transfer learning work broadly.



Same word, different vectors – because the network reads the context.

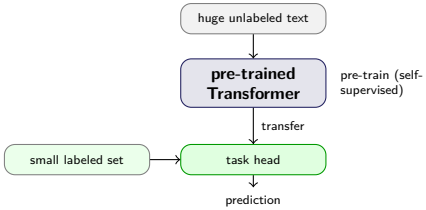
The now-standard NLP recipe is **two stages**:

1. Pre-train a large Transformer on *huge unlabeled* text with a self-supervised objective:

- masked-token prediction (BERT), or next-token prediction (GPT).
- Learns general-purpose language representations – expensive, but done *once*.

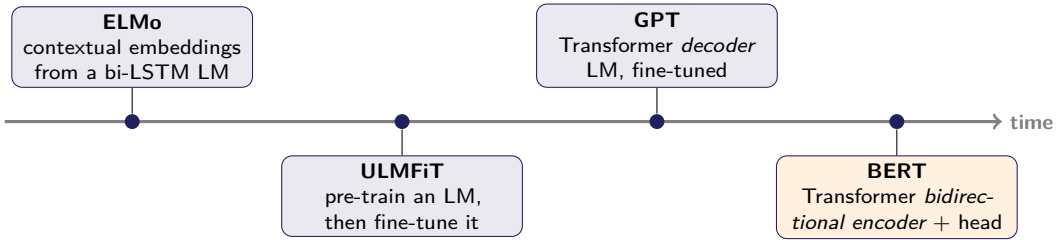
2. Transfer to your task with a *small labeled* dataset, two ways:

- **Feature extraction**: freeze the Transformer, feed its contextual outputs to a small trainable head.
- **Fine-tuning**: attach a head and update the whole network on the task (usually best when you have enough data).



Pre-train once on unlabeled text; adapt cheaply per task.

In barely a year (around **2018**) a wave of models made "pre-train once, adapt cheaply" the default recipe for NLP:



Two shifts along the way:

- from a pre-trained model as *fixed features* (ELMo) to **fine-tuning the whole model** (ULMFiT / GPT / BERT);
- from *LSTMs* to **Transformers**.

Why it mattered:

- one pre-trained model → strong results on many tasks with *little* labeled data;
- "download a model, add a head, fine-tune" became the default.

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

To some extent, any visual task can be solved now by:

- Construct a large-scale dataset labelled for that task.
- Specify a training loss and neural network architecture.
- Train the network and deploy.

To some extent, any visual task can be solved now by:

- Construct a large-scale dataset labelled for that task.
- Specify a training loss and neural network architecture.
- Train the network and deploy.

Issues:

- Expense of producing a new dataset for each new task.
- Some areas are supervision-starved, e.g. medical data, where it is hard to obtain annotation.
- Untapped/availability of vast numbers of unlabelled images/videos.

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

Medical Image Classification

Assume you are hired by a hospital to design a system to detect prostate cancer in CT images.

The hospital has provided you with a small labelled dataset of 600 CT scans (both benign and malignant).

The hospital has also given you access to their imaging archives which had **many more CT images**. However these images are **not labelled** and it will take considerable resources to annotate them (benign and malignant).

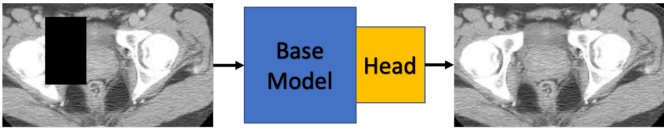


Not enough images to train from scratch.

Transfer learning is not an option here are we do not have large dataset from a similar task (supervised) and no pre-trained models.

- **Step 1**: Design a proxy task that does not require manual labelling.
- **Step 2**: Train a base model on the proxy task.
- **Step 3**: Do transfer learning with small labelled dataset.

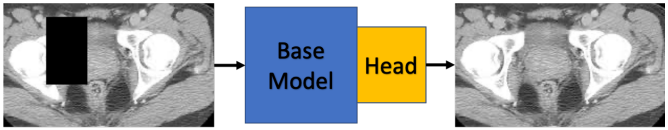
The proxy task has to be designed in a way that the model will capture information relevant to the task. This involved lots of engineering and we will not go into detail in this course.



Proxy Task: Randomly blackout a small region in a CT scan and try to recreate the original scan (with no blackout).

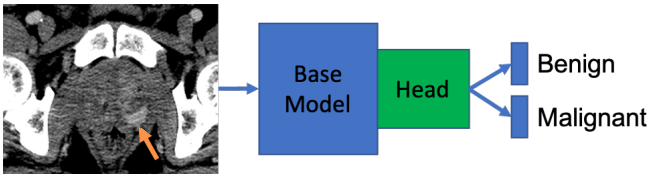
The unlabeled dataset can now be used to create a labeled dataset for the proxy task automatically (at no expense).

This proxy task is called “inpainting”.



Proxy Task: Randomly blackout a small region in a CT scan and try to recreate the original scan (with no blackout).

Intuition: To do the proxy task the network must learn about the structure of the CT scans.



Intended Task: Transfer and Train the model with small supervised dataset. Fine tune if necessary.

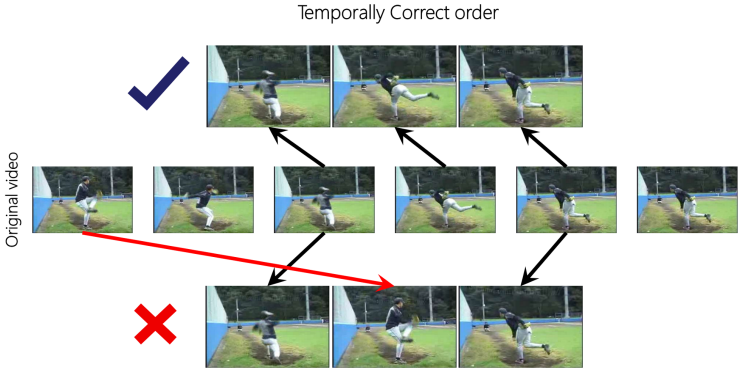
- A form of unsupervised learning where the data provides the supervision.
- In general, withhold some information about the data, and task the network with predicting it.
- The task defines a proxy loss, and the network is forced to learn what we really care about, e.g. a semantic representation, in order to solve it.

Many self-supervised tasks for images, videos.

Which image has the correct rotation?



Unsupervised Representation Learning by Predicting Image Rotations



Slide credit: Ishan Misra

Shuffle and Learn: Unsupervised Learning using Temporal Order Verification

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

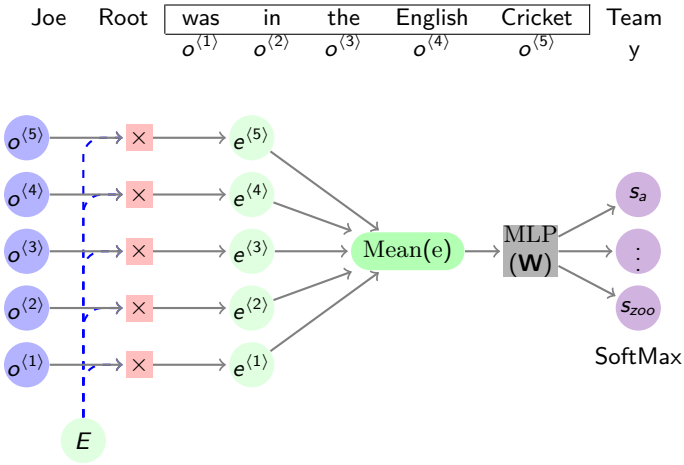
Learn by language modelling. E.g. Predict Next word:

Joe Root was in the English cricket ???.

Can create a supervised learning task where **x** is a fixed length sequence (window selected by user) and y is the next word in the sequence.

Joe	Root	was	in	the	English	Cricket	Team
		$x^{\langle 1 \rangle}$	$x^{\langle 2 \rangle}$	$x^{\langle 3 \rangle}$	$x^{\langle 4 \rangle}$	$x^{\langle 5 \rangle}$	y

A Neural Probabilistic Language Model



Train for parameters E and the weights of the MLP ($\mathbf{W}$) via back-propagation.

Joe	Root	was	in	the	English	Cricket	Team
		$x^{\langle 1 \rangle}$	$x^{\langle 2 \rangle}$	$x^{\langle 3 \rangle}$	$x^{\langle 4 \rangle}$	$x^{\langle 5 \rangle}$	y
				context			target

Context can be:

- Last 5 words.
- 5 words before and 5 words after.
- Last one word.
- Near by word.

Word2Vec and GloVe are more recent practical word embedding learning techniques.

Yann Lecun's Cake Theory at NIPS 2016



- **"Pure" Reinforcement Learning** (cherry)
  - The machine predicts a scalar reward given once in a while.
  - **A few bits for some samples**
- **Supervised Learning** (icing)
  - The machine predicts a category or a few numbers for each input
  - Predicting human-supplied data
  - **10→10,000 bits per sample**
- **Unsupervised/Predictive Learning** (cake)
  - The machine predicts any part of its input for any observed part.
  - Predicts future frames in videos
  - **Millions of bits per sample**
- **(Yes, I know, this picture is slightly offensive to RL folks. But I'll make it up)**

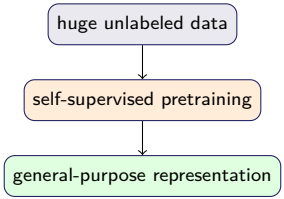


Yann LeCun is a co-recipient of the **2018 Turing Award** for his work in deep learning.

Self-supervision at scale

The word-embedding idea – *predict part of the input from the rest* – is the seed of modern self-supervision. Scaled up, two families dominate:

- **Masked / denoising**: hide part of the input and reconstruct it.
  - Text: masked-token prediction (BERT), next-token prediction (GPT).
  - Images: masked-patch prediction (MAE).
- **Contrastive**: pull together two views of the same item, push apart different items (SimCLR); across modalities, match images to their captions (**CLIP**).



No human labels are needed – the "labels" come from the data itself. This is what makes training at massive scale possible.

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

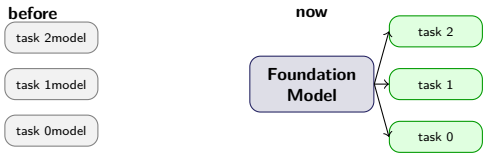
What is a Foundation Model?

A **foundation model** is a single model *pre-trained on broad data at scale* (usually self-supervised) that can be *adapted* to a wide range of downstream tasks.

The shift:

- **Before**: one bespoke model per task, each needing its own labeled dataset.
- **Now**: one foundation model $\rightarrow$ adapt to many tasks with little or no extra data.

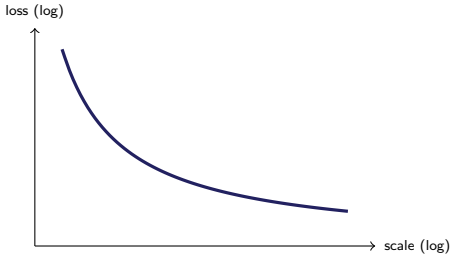
Examples: BERT, GPT, CLIP, ViT, Llama, Whisper, SAM.



One backbone, many downstream uses.

Why do foundation models dominate?

- **Scale.** More data, parameters and compute yield predictable gains – captured by **scaling laws** (loss falls as a power law in each).
- **Emergent abilities.** Beyond some scale, models show capabilities that were absent in smaller ones (e.g. in-context learning).
- **Amortised cost.** Pre-train once (expensive), adapt many times (cheap) – the cost is shared across every downstream task.
- **Generality.** One representation transfers across tasks and even modalities.



Scaling law: loss decreases smoothly as a power of scale – bigger reliably helps.

The same recipe – self-supervised pre-training of a large Transformer on broad data – recurs everywhere:

Model	Modality	Pre-training signal	Typical use
BERT	text	masked-token prediction	understanding, classification, NER
GPT / Llama	text	next-token prediction	generation, chat, reasoning
ViT / MAE	images	supervised / masked patches	image classification, backbone
CLIP	text + images	image–caption matching	**zero-shot** classification, retrieval
Whisper	audio	weakly-supervised transcription	speech recognition

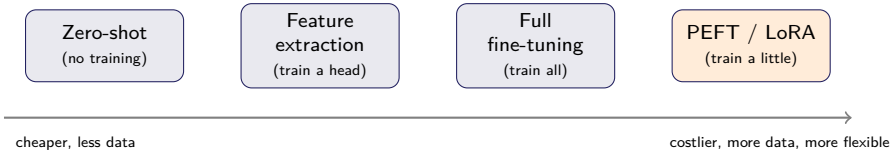
- **CLIP** is worth noting: by aligning images and text in one space, it can classify images it was never explicitly trained to label – our lead example for *zero-shot* adaptation next.

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

The adaptation spectrum

How much of the model do we touch? From changing *nothing* to changing *everything*:

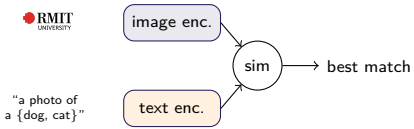


Zero-shot adaptation

Use the pre-trained model **as-is**, with no gradient updates.

- **Language**: describe the task in a *prompt*; the model responds. Giving a few examples in the prompt is *in-context (few-shot) learning* – still no weight updates.
- **Vision–language (CLIP)**: embed the image and a set of text prompts ("a photo of a {class}"); pick the class whose text embedding is closest.

Pros: no training, no labels, instant new tasks.
Cons: sensitive to prompt wording; usually below a tuned model.

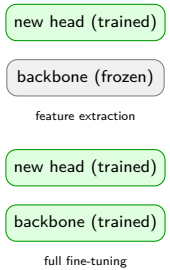


CLIP zero-shot: no dog/cat training needed.

Feature extraction and fine-tuning (recap)

You have already met these two for CNNs (e.g. ResNet):

- **Feature extraction**: *freeze* the backbone, use its output as fixed features, train only a small head (a "linear probe"). Fast, few parameters, good for small data.
- **Full fine-tuning**: *unfreeze* and update all weights on the new task. Best accuracy when you have enough data, but:
  - expensive (a full model + optimizer state per task),
  - can **overfit** small datasets,
  - risks **catastrophic forgetting** of pre-trained knowledge.



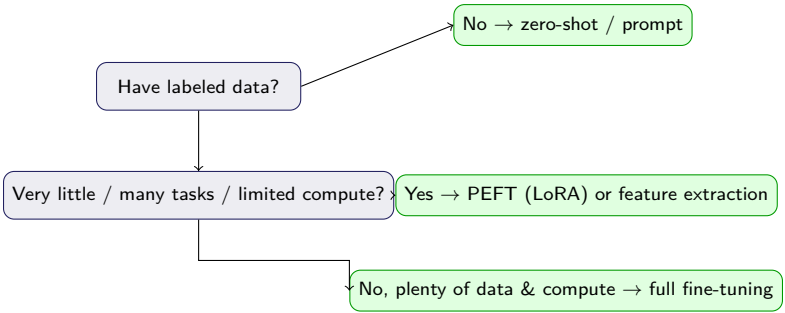
Green = updated, grey = frozen.

Choosing a strategy: trade-offs

Strategy	Labeled data	Train compute	Storage / task	Task performance
Zero-shot	none	none	none (shared model)	lowest
Feature extraction	small	low	tiny (a head)	moderate
Full fine-tuning	medium–large	high	**full copy**	highest
PEFT / LoRA	small–medium	low–medium	**tiny** (adapter)	near fine-tuning

- No single winner – the right choice depends on data, compute, how many tasks you serve, and latency.
- **PEFT** is attractive because it reaches close to full-fine-tuning quality while keeping the storage and compute of a light method.

Which strategy should I use?



A rule of thumb, not a law – always validate empirically (which is exactly what the lab has you do).

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

Why parameter-efficient fine-tuning (PEFT)?

Full fine-tuning of a large model is costly in ways that add up:

- You store a **full copy of every weight** for *each* task – billions of parameters per deployment.
- Training needs memory for gradients *and* optimizer state (e.g. Adam keeps two extra tensors per weight).
- Serving many tasks means loading many full models.

PEFT idea: freeze the pre-trained weights and train only a **small number of new parameters**. You keep almost all the benefit of fine-tuning at a tiny fraction of the cost.

Common PEFT methods:

- **Adapters**: small bottleneck layers inserted between blocks.
- **Prompt / prefix tuning**: learn a few "virtual token" vectors, leave the model untouched.
- **LoRA**: learn a low-rank update to existing weight matrices *(our focus, and the lab)*.

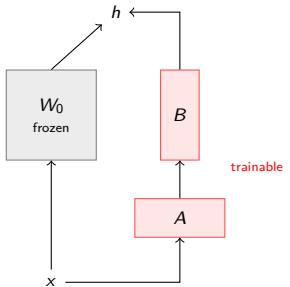
LoRA: Low-Rank Adaptation

Fine-tuning changes a weight matrix $W_0 \in \mathbb{R}^{d \times k}$ by some update ΔW. The key observation: this update has **low intrinsic rank**. So constrain it to be low-rank:

$$W = W_0 + \Delta W = W_0 + BA,$$

with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ and rank $r \ll \min(d, k)$.

- **Freeze** W_0; train only A and B.
- Parameters drop from $d \times k$ to $r(d + k)$ – often $< 1\%$ of the original.
- Forward pass: $h = W_0 x + BAx$ (scaled by α/r).



A small low-rank "side path" added to a frozen weight.

LoRA in practice

Key knobs:

- **rank** r: capacity of the update (e.g. 4–64).
- α: scaling of the update (α/r).
- **target modules**: usually the attention projections (W_q, W_v).

Why it's convenient:

- Adapters are tiny – ship a few MB per task, swap them in and out.
- At inference you can **merge** BA into W_0, so there is *no extra latency*.
- **QLoRA**: keep the frozen model in 4-bit and train LoRA on top – fine-tune large models on a single GPU.

```python
from peft import LoraConfig, get_peft_model

cfg = LoraConfig(
    r=8, lora_alpha=16,
    target_modules=["q_proj","v_proj"],
    lora_dropout=0.05,
    task_type="SEQ_CLS")

model = get_peft_model(base_model, cfg)
model.print_trainable_parameters()
# trainable: ~0.3% of all parameters
```

The `peft` library wraps a frozen base model with LoRA layers – exactly what you use in the lab.

Outline

1. Transfer learning
   - Image classification
   - Text classification
2. Self-Supervised learning
   - Image classification
   - Word Representation Learning
3. Foundation Models
4. Adapting Foundation Models
5. Parameter-Efficient Fine-Tuning
6. Limitations, Failure Cases & Bias

Foundation models fail too

Powerful, but not magic. Common failure modes:

- **Hallucination**: generating fluent but false content.
- **Distribution shift**: accuracy drops on data unlike the pre-training distribution.
- **Shortcut / spurious correlations**: relying on background or artefacts instead of the true signal.
- **Prompt sensitivity**: small wording changes flip zero-shot predictions.

Why this matters for adaptation:

- Zero-shot inherits *all* of the base model's blind spots.
- Fine-tuning on a small, biased dataset can *amplify* a shortcut.
- Strong average accuracy can hide poor performance on rare but important cases.

Always evaluate on *your* distribution and on the sub-groups you care about – not just an aggregate score.

Bias and responsible use

Foundation models are trained on large, uncurated web data, so they **absorb and can reproduce the biases** in it:

- **Representational harm**: stereotyped or skewed associations.
- **Allocative harm**: unequal error rates across groups in a downstream decision.
- Bias in the pre-trained model **propagates** into every task adapted from it.

Mitigations (partial):

- Evaluate with **disaggregated** metrics across groups.
- Curate / balance adaptation data; document data and models.
- Red-teaming, guardrails, human oversight for high-stakes use.

Also weigh the **compute and energy cost** of large-scale training and serving.

Summary

How to build capable models without large labeled datasets:

- **Representation learning**: transfer learning and self-supervision give general, reusable features.
- **Foundation models**: one large self-supervised model, adapted to many tasks – dominant because of scale, emergence and amortised cost.
- **Adaptation spectrum**: zero-shot $\rightarrow$ feature extraction $\rightarrow$ full fine-tuning $\rightarrow$ **PEFT**; pick by data, compute, #tasks and latency.
- **LoRA**: a low-rank, frozen-backbone update – near fine-tuning quality at a tiny cost.
- **Limitations**: hallucination, shift, shortcuts, and inherited **bias** – evaluate carefully.

Lab: adapt one foundation model four ways – zero-shot, feature extraction, fine-tuning and **LoRA** (via `peft`) – and compare accuracy, cost and failure cases.